MONASH UNIVERSITY

Faculty of Information Technology

FIT5032 Internet Applications Development

Assessed Lab 9

Cloud Functions

Alibaba Cloud Function Compute and Firestore

Student Name	Han Wang
Student ID	36668664
Tutorial/Lab	Tuesday 2:00 pm
Project Name	hwang-library
Git Branch	assessed-lab-9
GitHub Repository	https://github.com/kokyouu/hwang-library

31 July 2026

1. Overview

This report documents the completed FIT5032 Assessed Lab 9 implementation in the existing `hwang-library` Vue 3 project. The solution uses one isolated Alibaba Cloud Function Compute web function to read the existing Cloud Firestore books collection. The same function exposes a count endpoint and the Distinction/High Distinction marketplace endpoint, while Cloudflare Pages hosts only the static Vue client.

Assessment item	Completed result
Cloud function	Node.js 20 custom-runtime web function deployed in China (Hong Kong)
Book count	Live Firestore count returned through /api/books/count
Vue integration	Book Counter displays 3 records and the Alibaba Cloud source
Extension task	Marketplace prices and displays the live Firestore catalogue
Deployment	Static preview deployed from GitHub commit eeb790c
Version control	Dedicated assessed-lab-9 branch with focused commits

2. Architecture and Resource Isolation

The browser loads the Vue production build from Cloudflare Pages. Vue sends GET requests to the Function Compute public HTTPS trigger. The function reads the Firestore REST API and returns JSON with CORS headers. No existing Alibaba Cloud ECS, VPC, security group, load balancer, domain, NAS or OSS resource is attached to this Lab 9 function.

Component	Configuration
Vue client	https://b8fb1089.hwang-library.pages.dev
Cloud API	https://fit-lab-library-zpfjrylreg.cn-hongkong.fcapp.run
Function	fit5032-lab9-hwang-library
Region/runtime	cn-hongkong; Node.js 20 custom runtime on Debian 11
Elasticity	0 minimum instances; scales to zero when idle
Network	Default public egress only; VPC access disabled
Data source	Cloud Firestore project hwang-library-lab7, collection books

2.1 Deployed function evidence

The Function Compute detail page identifies the account, region, function, HTTP trigger, editable LATEST version and Debian 11 custom runtime in one deployment view.

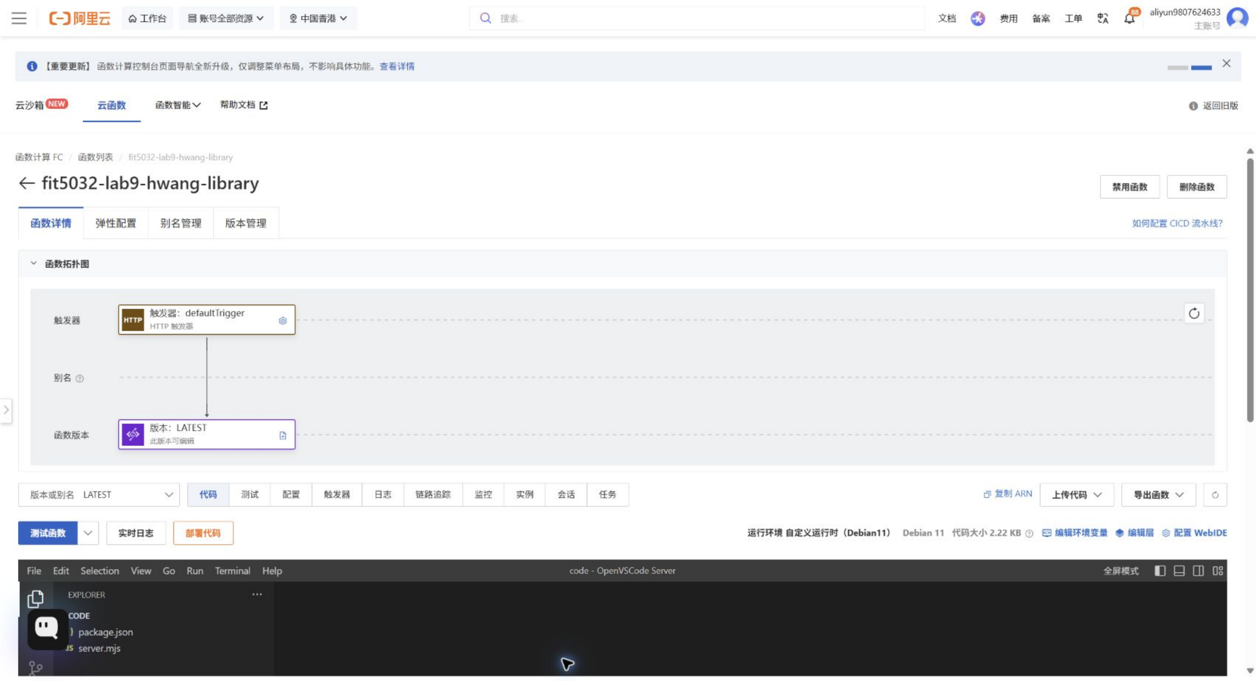

Figure 1. Alibaba Cloud account, Hong Kong region, isolated function name, HTTP trigger and Node.js custom runtime.

3. Alibaba Cloud Function Configuration

3.1 Public HTTP trigger

The default HTTP trigger supports GET, HEAD and OPTIONS requests and exposes the function through HTTPS. Authentication is set to No Authentication because the assessed Vue application is a public client and cannot sign requests with an Alibaba Cloud access key. The function itself accepts only GET and OPTIONS and returns 405 for other methods.

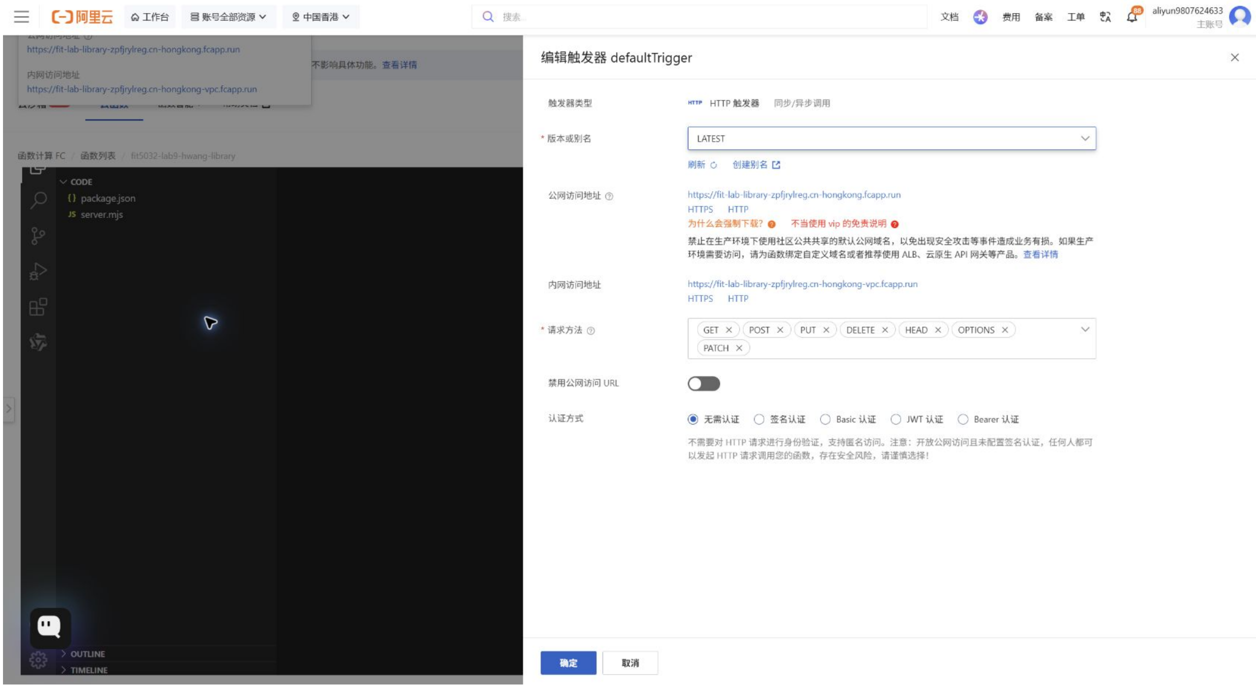

Figure 2. Public Function Compute URL and the selected No Authentication HTTP-trigger setting.

3.2 Scale-to-zero cost control

No elastic policy or reserved instance is configured. Alibaba Cloud therefore uses elastic instances with a minimum instance count of 0. This avoids an always-on server and keeps the short lab verification within the approved low-cost limit.

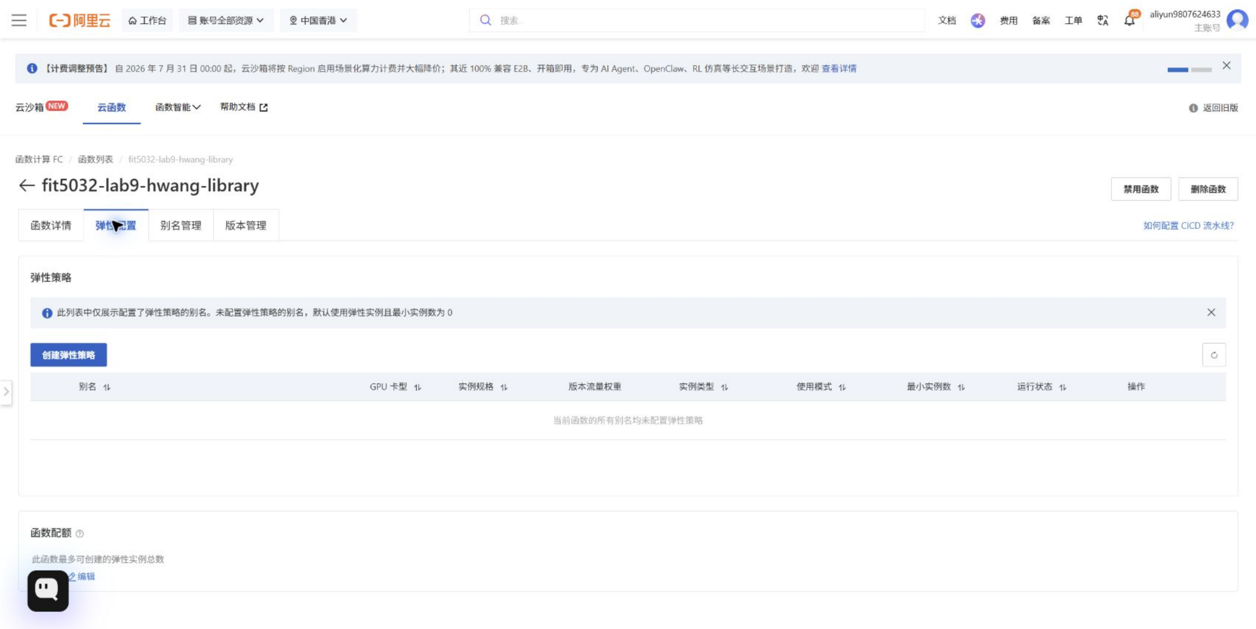

Figure 3. Elastic configuration confirming the default minimum instance count is 0.

4. Cloud Function Implementation

4.1 Firestore book count

The count handler retrieves the live books collection, calculates the document count and returns the collection, project and source metadata. The response identifies Alibaba Cloud Function Compute so the browser result can prove which cloud platform executed the request.

Firestore count cloud function
aliyun-function/server.mjs

```
async function countBooks(fetchImpl) {
  const books = await fetchBooks(fetchImpl)
  return jsonResponse({
    success: true,
    count: books.length,
    collection: 'books',
    source: 'Cloud Firestore via Alibaba Cloud Function Compute',
    projectId: FIREBASE_PROJECT_ID,
    generatedAt: new Date().toISOString(),
  })
}

```

Figure 4. Count route returning the live Firestore document count and cloud-source metadata.

4.2 Marketplace extension

The marketplace handler calculates a catalogue value from the current number of records, generates Snapshot and Semester Feed offers, and attaches a per-record educational licence price. Values are derived at request time, so additions to Firestore automatically change the catalogue and offers.

Marketplace response generated from live records
aliyun-function/server.mjs

```
async function buildMarketplace(fetchImpl) {
  const books = await fetchBooks(fetchImpl)
  const recordPriceAud = 4.5
  const catalogueValueAud = Number((books.length * recordPriceAud).toFixed(2))

  return jsonResponse({
    success: true,
    product: {
      name: 'Hwang Library Curated Books Dataset',
      edition: `Firestore live edition - ${books.length} records`,
      currency: 'AUD',
      catalogueValueAud,
      licence: 'Single-project educational data licence',
    },
    offers: [
      {
        id: 'snapshot',
        name: 'Snapshot',
        priceAud: catalogueValueAud,
        description: 'One JSON export of the current Firestore collection.',
      },
      {
        id: 'semester',
        name: 'Semester Feed',
        priceAud: Number((catalogueValueAud * 2.4).toFixed(2)),
        description: 'Live API access for one teaching semester.',
      },
    ],
    records: books.map((book) => ({ ...book, recordPriceAud })),
    source: 'Cloud Firestore via Alibaba Cloud Function Compute',
    generatedAt: new Date().toISOString(),
  })
}

```

Figure 5. Marketplace product, licence offers and priced live records.

4.3 HTTP routing and error handling

A small request router handles CORS preflight, rejects unsupported methods, provides a health endpoint and dispatches the two assessed API paths. Unexpected failures are returned as JSON with HTTP status 500.

HTTP routing, health check and error handling
aliyun-function/server.mjs

```
export async function handleRequest(request, fetchImpl = fetch) {
  if (request.method === 'OPTIONS') {
    return new Response(null, { status: 204, headers: corsHeaders })
  }

  if (request.method !== 'GET') {
    return jsonResponse({ success: false, error: 'Method not allowed.' }, 405)
  }

  const { pathname } = new URL(request.url)

  try {
    if (pathname === '/health') {
      return jsonResponse({
        success: true,
        service: 'fit5032-lab9-hwang-library',
        platform: 'Alibaba Cloud Function Compute',
      })
    }
    if (pathname === '/api/books/count') return await countBooks(fetchImpl)
    if (pathname === '/api/books/marketplace') return await buildMarketplace(fetchImpl)
    return jsonResponse({ success: false, error: 'Route not found.' }, 404)
  } catch (error) {
    return jsonResponse(
      {
        success: false,
        error: error instanceof Error ? error.message : 'Unable to process the request.',
      },
      500,
    )
  }
}

```

Figure 6. Health, count and marketplace routes with method and error handling.

5. Vue Results

5.1 Book Counter

The production Vue build reads the Function Compute base URL from `.env.production`. Selecting Get Book Count calls the Alibaba endpoint and displays the live count, collection, Firebase project and cloud source. The verified result is 3.

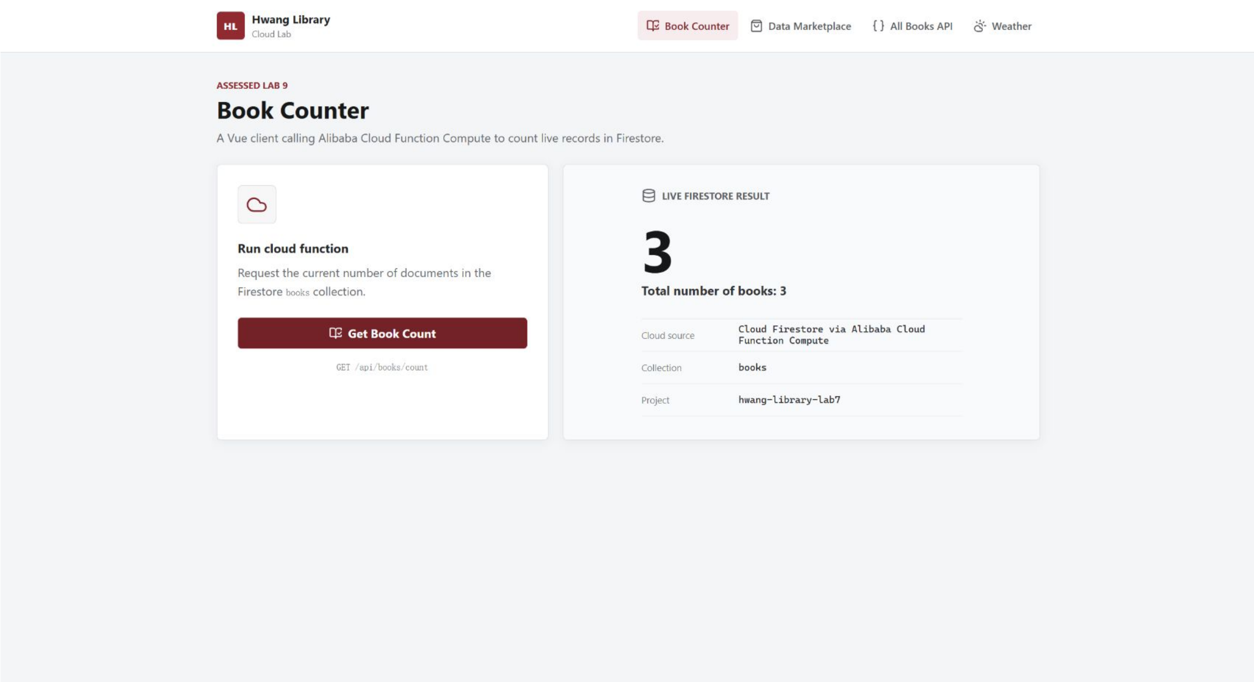

Figure 7. Online Book Counter showing 3 live Firestore records from Alibaba Cloud Function Compute.

5.2 Book Data Marketplace

The marketplace page loads the second function route and presents the data as a commercial teaching dataset. The verified response contains three books, a catalogue value of AUD 13.50, Snapshot and Semester Feed offers, and the Alibaba Cloud source label.

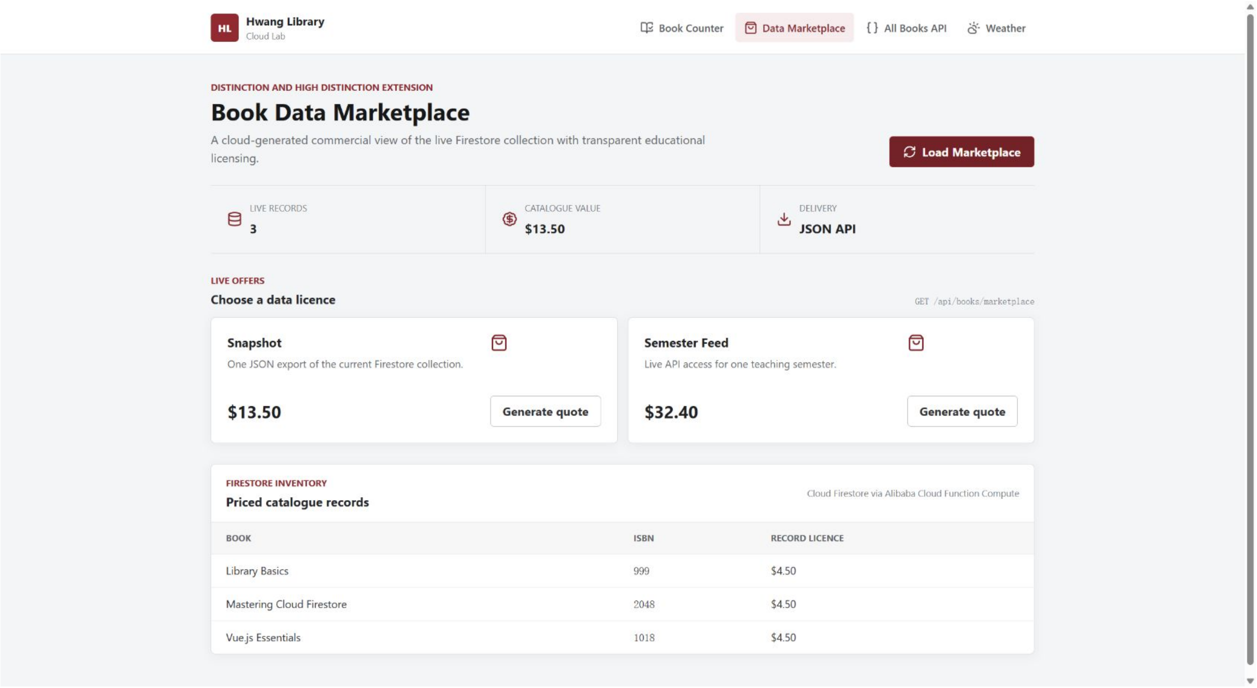

Figure 8. Online marketplace with live offers and all three Firestore records.

6. Deployment and Verification

GitHub commit `eeb790c` triggered a successful Cloudflare Pages preview deployment. Cloudflare serves only the static application; all Lab 9 API requests are sent to the Alibaba Cloud Function Compute public URL.

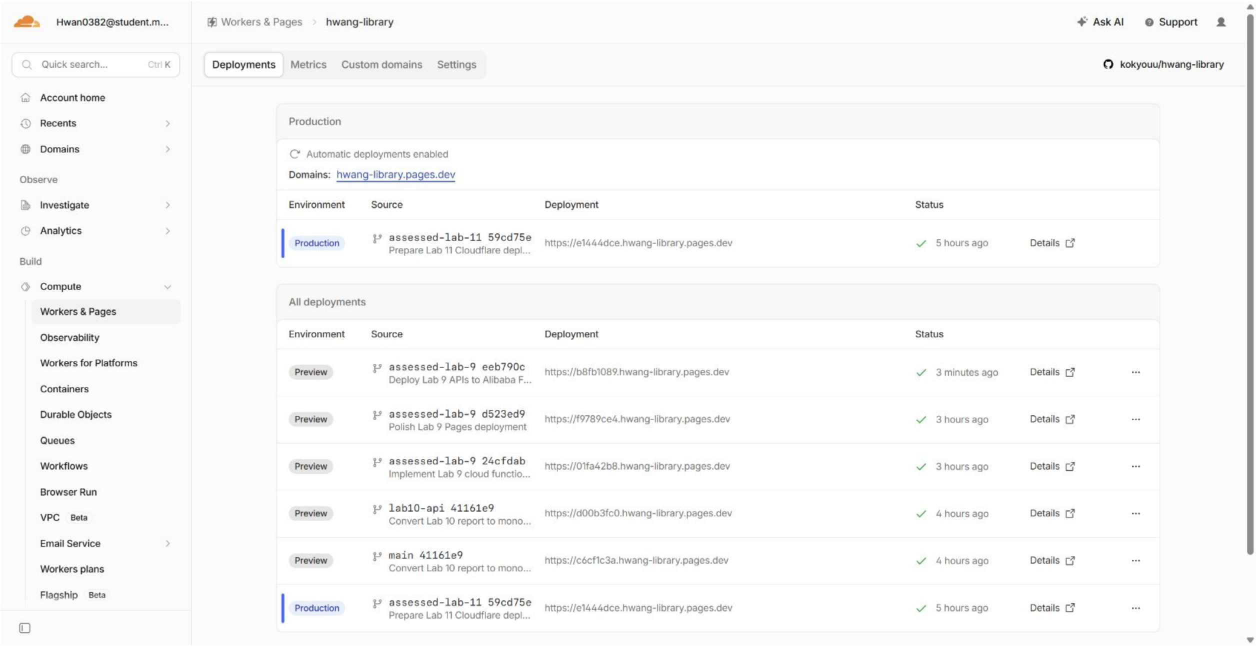

Figure 9. Successful preview deployment for commit eeb790c on assessed-lab-9.

6.1 Test results

Verification	Result
GET /health	HTTP 200; Alibaba Cloud platform identified
GET /api/books/count	HTTP 200; count 3; Firestore project and collection confirmed
GET /api/books/marketplace	HTTP 200; 3 records and two offers
Function tests	3 passed
Vue unit tests	6 passed across 4 test files
TypeScript	vue-tsc completed without errors
Production build	Vite build completed successfully
Online browser	Book Counter and Marketplace loaded Alibaba data successfully

7. Git Version Control

All work was completed on the dedicated `assessed-lab-9` branch. The existing main branch and unrelated lab branches were not modified. The implementation commit was pushed to the requested GitHub repository before this report was generated.

Git commit history
branch: assessed-lab-9

```
* f35d65b (HEAD -> assessed-lab-9, origin/assessed-lab-9) Restore color screenshots in Lab 9 report
* 2e8e652 Add monochrome Lab 9 report and evidence
* eeb790c Deploy Lab 9 APIs to Alibaba Function Compute
* d523ed9 Polish Lab 9 Pages deployment
* 24cfdab Implement Lab 9 cloud function workflows
* 41161e9 (origin/main, origin/lab10-api, origin/HEAD, main) Convert Lab 10 report to monochrome
* cd4f9c8 Make report headings black
* cb9d367 Add current-location evidence to Lab 10 report
```

Figure 10. Local Git history for the assessed-lab-9 branch.

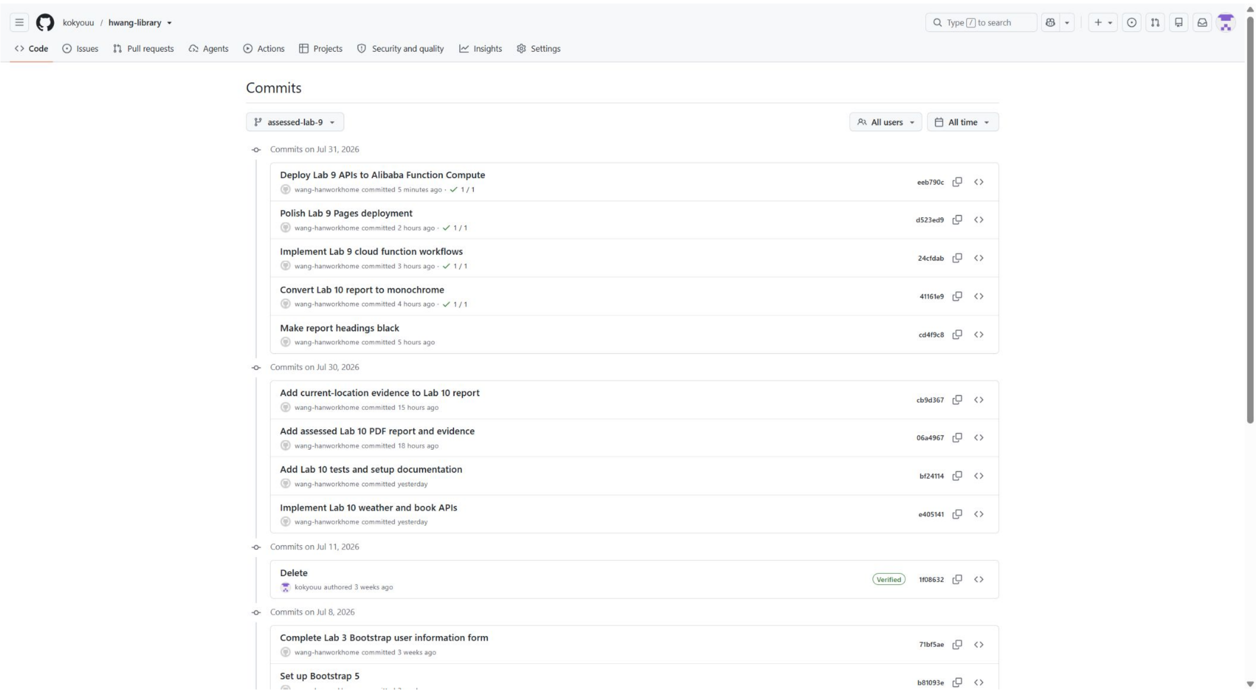

Figure 11. GitHub commit history showing the pushed Alibaba Cloud implementation commit.

8. Conclusion

The Assessed Lab 9 requirements are complete. The application counts live Firestore documents through an Alibaba Cloud web function, displays the result in Vue, provides the marketplace extension, passes automated and browser verification, and is deployed from a dedicated Git branch. The Alibaba resource is isolated, uses no existing server or VPC, and scales to zero when idle.